

Atividade 3.15: Medir o efeito da representação

Laura Barbosa Henrique

17 de setembro de 2026

Pergunta

Qual das duas representações executa `contar_triangulos` mais rápido, e a resposta depende da densidade do grafo?

Método

As duas implementações seguem a interface `Grafo` do capítulo, escritas do zero. Os grafos aleatórios têm $n = 2000$ e foram gerados percorrendo os pares $u < v$ e inserindo cada um com probabilidade δ , com `random.seed(42)` fixada antes do laço. A semente é fixada dentro da função geradora, então as duas implementações recebem exatamente o mesmo grafo em cada densidade, e a comparação isola a estrutura.

O tempo é a mediana de três execuções de `contar_triangulos` com `time.perf_counter`. O espaço é contado em posições ocupadas pela estrutura: n^2 na matriz e $n + 2m$ na lista. **Ambiente.** As medições de tempo e os experimentos foram executados no Google Colab com Python 3.14.0. Tempo depende da máquina, então os valores absolutos da tabela só se comparam entre si, dentro da mesma execução. O que se compara entre máquinas é a razão entre as duas implementações.

Conferências (itens 2 e 3)

O teste do item 2 constrói o grafo da Figura 3.1 nas duas implementações e passa nas quatro verificações em ambas: $n = 5$, $m = 6$, sequência de graus $(3,3,2,2,2)$ e soma dos graus igual a $2m = 12$. Um quinto teste compara as vizinhanças vértice a vértice e confirma que as duas implementações descrevem o mesmo grafo.

A matriz de incidência do item 3, com a numeração de arestas da figura:

$$B(G) = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}$$

As seis colunas somam 2, e as linhas somam 2, 3, 3, 2, 2, que são os graus de v_1 a v_5 . O total é $12 = 2m$.

Medição (itens 4 e 5)

δ alvo	Implementação	m	δ obtida	Tempo Mediano (s)	Espaço (posições)
0,001	Grafo Matriz	1.910	0,000955	0,3273	4.000.000
0,001	Grafo Lista	1.910	0,000955	0,0008	5.820
0,05	Grafo Matriz	99.819	0,049934	8,9697	4.000.000
0,05	Grafo Lista	99.819	0,049934	0,5103	201.638
0,5	Grafo Matriz	999.109	0,499804	138,9812	4.000.000
0,5	Grafo Lista	999.109	0,499804	49,5932	2.000.218

A contagem de triângulos coincidiu nas duas implementações em cada densidade (2, 165.360 e 166.216.758), o que confirma que a diferença de tempo é da estrutura e não do algoritmo.

Resposta ao item 6

A lista foi mais rápida nas três densidades, e a ordem entre elas não muda. O que muda é o tamanho da vantagem, que cai de forma acentuada conforme δ cresce: a matriz é cerca de 409 vezes mais lenta em $\delta \approx 0,001$, 17,5 vezes em $\delta \approx 0,05$, e 2,8 vezes em $\delta \approx 0,5$.

Isso é o que a composição de custo da seção 3.11.1 prevê. O algoritmo é dominado por percurso de vizinhança, e é exatamente aí que as duas diferem: vizinhos custa $\Theta(d(v))$ na lista e $\Theta(n)$ na matriz, porque a matriz varre a linha inteira mesmo quando o vértice tem poucos vizinhos. O teste de adjacência, única operação em que a matriz ganha, é usado uma vez por trio e não compensa. Quando δ cresce, $d(v)$ se aproxima de n e as duas varreduras se aproximam, o que explica a vantagem encolhendo.

Para comparar previsão e medida em número, os passos do laço interno foram contados exatamente sobre cada grafo gerado, sem executar o laço triplo: $\sum_v d(v) |\{u \in N(v) : u < v\}|$ na lista, e $n^2 + mn$ na matriz.

δ	Passos (lista)	Passos (matriz)	Razão prevista	Razão medida	ns/passos (L)	ns/passos (M)
0,001	9.426	7.820.000	829,6	409,0	84,8	41,8
0,05	10.256.628	203.638.000	19,9	17,5	49,7	43,9
0,5	1.000.896.459	2.002.218.000	2,0	2,8	49,6	69,4

Onde a medida concorda. O sentido da comparação e a tendência concordam nas três densidades, e em $\delta \approx 0,5$ a concordância é quantitativa: previsto 2,0, medido 2,8. Esse fator tem origem clara, é o $n/d(v)$ da estrutura: com $\delta \approx 0,5$, cada vértice tem cerca de 999 vizinhos, e a matriz varre 2000 posições para entregá-los.

Onde não concorda, e por quê. Em $\delta \approx 0,001$ a razão prevista é 829,6 e a medida é 409,0, duas vezes menor. A explicação proposta é que a contagem de passos ignora custo fixo por chamada, e nessa densidade o custo fixo domina a lista: são apenas 9.426 passos de laço interno contra 2000 iterações do laço externo e mais de 5000 iteradores criados, tudo $\Theta(n)$ e independente de δ . Uma segunda discordância aparece na matriz, no sentido oposto: seu custo por passo sobe de 41,8 para 69,4 ns. A explicação proposta é que uma posição varrida que vale 1 custa mais que uma que vale 0, porque produz um valor e faz o corpo do laço executar o filtro $w < v$ e **tem_aresta**. Em densidade baixa quase toda posição varrida vale 0 e sai barato; em $\delta \approx 0,5$, metade delas produz trabalho.

Item 7: conjunto real (email-Eu-core)

O conjunto escolhido é o `email-Eu-core`, trocas de e-mail entre membros de uma instituição de pesquisa europeia, distribuído pelo SNAP em lista de arestas de duas colunas. Cada linha registra que a pessoa u enviou pelo menos um e-mail para v . Foi lido com `ler_pares` e `construir` e conferido com `conferir`, sem alteração no código do capítulo.

Medida	Valor
Linhas do arquivo	25.571
Linhas ignoradas	0
Laços descartados	642
Pares entregues	24.929
Arestas repetidas	8.865
n (ordem)	986
Rótulos distintos no arquivo	1.005
m (tamanho)	16.064
Soma dos graus	32.128
Aperto de mão ($2m$)	confere
Vértices isolados	0
$\delta(G)$	0,03308

Ordem contra rótulos distintos: os dois números não coincidem, e isso é informação sobre os dados. O arquivo tem 1.005 rótulos distintos, e o grafo construído tem ordem 986. A diferença de 19 vem dos 642 laços descartados na leitura: verificou-se que esses 19 rótulos aparecem apenas em linhas de laço, isto é, são pessoas que, no período coberto, só constam como tendo enviado e-mail a si mesmas. Como o capítulo trata de grafos simples, o laço sai, e com ele sai o vértice, que passa a não ter nenhuma aresta que o sustente.

Orientação: O arquivo descreve arcos, e não arestas. As 8.865 repetições foram decompostas, e a decomposição é limpa: nenhuma é o mesmo par ordenado aparecendo duas vezes, e todas as 8.865 são arco recíproco, ou seja, (v, u) chegando depois de (u, v) . Lido como grafo não orientado, os recíprocos colapsam e restam $m = 16.064$ arestas, contra os 24.929 arcos distintos. Logo, 8.865 das 16.064 arestas, ou 55,2%, têm os dois sentidos: mais da metade das relações de e-mail são respondidas (alta reciprocidade).

Conferência independente do grafo lido: O SNAP publica 105.461 triângulos para este conjunto. Executando `contar_triangulos` sobre o grafo construído, o resultado é exatamente 105.461. Isso valida de uma vez a leitura, a tradução de rótulos e a implementação, sem recorrer a biblioteca externa.

Representação escolhida: Lista de adjacência. A densidade medida é $\delta(G) = 0,03308$, com grau médio $2m/n \approx 32,6$ em 986 vértices. O valor cai entre as densidades 0,001 e 0,05 do experimento, mais perto da segunda, faixa em que a lista foi muito mais rápida. Em espaço, a matriz ocuparia $n^2 = 972.196$ posições contra $n + 2m = 33.114$ da lista, uma razão de 29 vezes.

Conclusão

Para este algoritmo, a lista de adjacência é a escolha em qualquer das densidades testadas, e a diferença deixa de ser decisiva apenas quando o grafo se aproxima do regime denso.

O espaço reforça a mesma escolha: em $\delta \approx 0,001$ a lista ocupa 5.820 posições contra 4.000.000 da matriz, e mesmo em $\delta \approx 0,5$ ela ainda ocupa metade. O conjunto real do item 7 tem $\delta(G) = 0,03308$ que cai dentro da faixa coberta pela medição, perto de $\delta \approx 0,05$. A decisão de representação para ele, portanto, não depende de suposição: está apoiada na densidade medida e no experimento dos itens 4 e 5.